

LIBXSTREAM Samples

Ozaki Scheme -- OpenCL

High-precision GEMM on OpenCL devices via mantissa slicing (Scheme 1) or Chinese Remainder Theorem (Scheme 2). Both schemes decompose FP matrices into int8/u8 tiles and use DPAS/XMX matrix engines when available. This is an OpenCL adaptation of the CPU-based Ozaki sample in LIBXS.

Build

```
cd samples/ozaki
make [GNU=1] [DBG=1]
```

Requires an OpenCL runtime and headers. BLAS is linked via `BLAS=2` for the reference GEMM.

Run

```
./ozaki.x [M [N [K [transa [transb [alpha [beta [lda [ldb [ldc]]]]]]]]]]]
```

All arguments are positional and optional:

Pos.	Argument	Default	Description		1	M	257	Rows of C and op(A)	2	N	M	Columns of C and op(B)	3	K	M	Inner dimension	4	transa	0	0=N, 1=T for A	5	transb	0	0=N, 1=T for B	6	alpha	1	Scalar multiplier for A*B	7	beta	1	Scalar multiplier for C	8	lda	auto	Leading dimension of A	9	ldb	auto	Leading dimension of B	10	ldc	M	Leading dimension of C	
------	----------	---------	-------------	--	---	---	-----	---------------------	---	---	---	------------------------	---	---	---	-----------------	---	--------	---	----------------	---	--------	---	----------------	---	-------	---	---------------------------	---	------	---	-------------------------	---	-----	------	------------------------	---	-----	------	------------------------	----	-----	---	------------------------	--

Environment Variables

Scheme Selection | Variable | Default | Description | |||| | `OZAKI` | 2 | 1=mantissa slicing, 2=CRT (default), 3=adaptive, 0=bypass BLAS | | `OZAKI_FP` | 64 | 64=fp64 (double), 32=fp32 (float) | | `OZAKI_N` | (auto) | Slices (Sch.1: fp64=8, fp32=4) or primes (Sch.2: fp64=16, fp32=9) |

`OZAKI=3` (adaptive) starts with Scheme 1 on the first call to learn the effective cutoff from preprocessing occupancy data. Subsequent calls compare the Scheme-1 pair count against the Scheme-2 prime count and pick the cheaper path. The cutoff is cached alongside the preprocessed buffers and reused on cache hits without any device-to- host readback.

Accuracy | Variable | Default | Description | |||| | `OZAKI_FLAGS` | 3 | Sch.1 bitmask: 1=Triangular, 2=Symmetrize, 0=full S². No Sch.2 | | `OZAKI_TRIM` | 0 | Precision levels to trim (0=exact). ~7 bits (Sch.1), ~4 bits (Sch.2) | | `OZAKI_I8` | 0 | Sch.2: use signed i8 residues (moduli<=128) instead of u8 | | `OZAKI_GROUPS` | 0 | Sch.2: K-grouping factor, consecutive K panels share reconstr. |

Hardware Control | Variable | Default | Description | |||| | `OZAKI_RTM` | (auto) | Register tiling M (power of two). Auto: 2 (HIER), 4 (256-GRF) | | `OZAKI_RTN` | (auto) | Register tiling N (power of two). Auto: 2 (Intel GPU), 1 (other) | | `OZAKI_WG` | 0 | Work-group size hint (0=no hint) | | `OZAKI_SG` | (auto) | Sub-group size (forced to 16 with XMX) | | `OZAKI_BIGGRF` | (auto) | Override 256-GRF detection (0=off, 1=on). HIER defaults to 128 | | `OZAKI_KU` | 2 | K-loop unroll factor | | `OZAKI_RC` | 8 | DPAS repeat count (8 or 4) | | `OZAKI_PB` | 1 | Sch.2: CRT prime batching factor | | `OZAKI_HIER` | (auto) | Sch.2: hierarchical CRT (default on). Two-level Garner reconstr. | | `OZAKI_PREFETCH` | 0 | Sch.1: enable prefetching | | `OZAKI_SCALAR_ACC` | 0 | Sch.1: force scalar accumulation |

Memory and Caching | Variable | Default | Description | |||| | `OZAKI_DEVPOOL` | 0 | Device memory pool via USM/SVM (eliminates per-call alloc overhead) | | `OZAKI_CACHE` | 0 | Preprocessing cache bitmask: 1=A, 2=B, 3=both. Skips on match |

The preprocessing cache also stores the last effective cutoff from Scheme 1 occupancy detection. On cache hits the cutoff is reused without device-to-host readback, eliminating the sync bubble.

Benchmark	Variable	Default	Description		NREPEAT	1	Number of benchmark repetitions
OZAKI_VERBOSE	0	0=silent, 1=errors, 2=warnings, 3+=all. Neg.=all					

Additional variables for profiling, accuracy monitoring, and complex GEMM dispatch (OZAKI_PROFILE, OZAKI_THRESHOLD, OZAKI_STAT, OZAKI_EPS, OZAKI_RSQ, OZAKI_EXIT, OZAKI_COMPLEX) are handled by the LIBXS Ozaki sample (LIBXS), which owns the GEMM interceptor. See its README for details.

Kernel Registry

Scheme 1 fused GEMM kernels are compiled on demand via a JIT registry. The compile-time cutoff (OZAKI_CUTOFF) is baked into each kernel specialization, allowing the compiler to eliminate dead slice-pair iterations and reduce register pressure. The first call with a given cutoff value triggers JIT compilation (~100 ms); subsequent calls hit the registry cache. Typical workloads produce 2-3 specializations (full cutoff, reduced cutoff, each with/without bounds checking).

Example

```
./ozaki.x 256
```

Scheme 2 on a large matrix:

```
OZAKI=2 ./ozaki.x 4096
```

Adaptive scheme selection with caching:

```
OZAKI=3 OZAKI_CACHE=3 ./ozaki.x 4096
```

Quick Tuning Guide

Scheme 2 (CRT, OZAKI=2, default): fixed cost of P integer GEMMs plus hierarchical Garner reconstruction. Predictable performance regardless of data distribution. Use OZAKI_GROUPS for K-grouping at large sizes. The hierarchical CRT (OZAKI_HIER, on by default) halves private residue arrays and enables GRF128 for doubled thread occupancy.

Scheme 1 (mantissa slicing, OZAKI=1): up to $S*(S+1)/2$ integer GEMMs, but adaptive cutoff can reduce this substantially for narrow exponent spans. Use OZAKI_TRIM to trade accuracy for speed.

Adaptive (OZAKI=3): automatically picks the cheaper scheme per call based on preprocessing occupancy. Best with OZAKI_CACHE=3 to avoid repeated occupancy readbacks.

Enable OZAKI_CACHE=3 when A or B stays constant across calls. Enable OZAKI_DEVPOOL=1 for repeated calls with similar sizes.

Small Matrix Multiplication (SMM) -- OpenCL

Batched small matrix multiplications (SMM) on OpenCL devices via the ACC LIBSMM interface. Originated from the DBCSR OpenCL backend, adapted to run on top of LIBXSTREAM. Includes GPU-accelerated kernels, a benchmark driver, and an auto-tuning framework.

Build

```
cd samples/smm
make [WITH_GPU=<device>] [ELEM_TYPE=<type>]
```

Produces acc_bench.x. Requires an OpenCL runtime, BLAS (BLAS=2), and LIBXS built from a sibling directory.

Make variable	Default	Description		ELEM_TYPE	double	Element precision: double or float	WITH_GPU
auto	Device for tuned parameters (PVC, A100, ...).	Fallback: all CSVs					

Benchmark Driver

```
./acc_bench.x [nrepeat [batchsize [M [N [K [nc [na [nb]]]]]]]]]
```

The kernel shape can also be given as $M \times N \times K$:

```
./acc_bench.x 5 30000 13x5x7
```

The first argument can be a file with one set of parameters per line. The batchsize argument accepts K/M/G suffixes for memory-budget mode.

Argument	Default	Description		nrepeat	66 (3+CHECK)	Number of timed repetitions	batchsize	30000
Matrix products per batch (stack size)	M	23	Rows of A and C	N	M	Columns of B and C	K	M
Columns of A / rows of B	nc	batchsize/16	Number of unique C-matrices	na	10nc	Number of unique A-matrices /	nb	10nc
Number of unique B-matrices								

Environment Variables (Benchmark)	Variable	Default	Description		CHECK	-1	Accuracy: negative=auto-threshold, 0=off, positive=custom	CHECK_H2D	-	Minimum H2D bandwidth (GB/s); fail if below	CHECK_DEV	-	Minimum device GFLOPS/s; fail if below	CHECK_HST	-	Minimum host GFLOPS/s; fail if below	DEVICE	0	Device index (multi-rank: device = rank % ndevices)	NREPEAT_H2D	1	H2D copy repetitions for bandwidth measurement	NREPEAT_SMM	1	SMM kernel launches per timed iteration (for profiling)	BATCHSIZE_SMM	-	Override batchsize (and optionally nrepeat: <code>bs,nrep</code>)
--	----------	---------	-------------	--	-------	----	---	-----------	---	---	-----------	---	--	-----------	---	--------------------------------------	--------	---	---	-------------	---	--	-------------	---	---	---------------	---	--

LIBSMM Kernel Parameters

OpenCL kernels are generated at runtime for any (M, N, K) shape within MAX_KERNEL_DIM. Tuned parameters can be embedded at build time or loaded at runtime for better performance.

Environment Variables (LIBSMM)	Transpose kernel:
---------------------------------------	-------------------

Variable	Default	Description		OPENCL_LIBSMM_TRANS_BUILDOPTS	-	Extra OpenCL build options	OPENCL_LIBSMM_TRANS_INPLACE	0	Non-zero: in-place transpose (no LDS)	OPENCL_LIBSMM_TRANS_BM	auto	Block size in M-direction (0 < BM <= M)
----------	---------	-------------	--	-------------------------------	---	----------------------------	-----------------------------	---	---------------------------------------	------------------------	------	---

Multiply kernel:

Variable	Default	Description		OPENCL_LIBSMM_SMM_BUILDOPTS	-	Extra OpenCL build options	OPENCL_LIBSMM_SMM_PARAMS	auto	Tuned parameters: 0=disable, or CSV path	OPENCL_LIBSMM_SMM_BS	auto	Intra-kernel mini-batchsize	OPENCL_LIBSMM_SMM_BM	auto	Block size in M-direction (0 < BM <= M)	OPENCL_LIBSMM_SMM_BN	auto	Block size in N-direction (0 < BN <= N)	OPENCL_LIBSMM_SMM_AP	auto	Access pattern for parameter/stack array	OPENCL_LIBSMM_SMM_AA	auto	Access pattern for A-matrix array	OPENCL_LIBSMM_SMM_AB	auto	Access pattern for B-matrix array	OPENCL_LIBSMM_SMM_AC	auto	Access pattern for C-matrix array
----------	---------	-------------	--	-----------------------------	---	----------------------------	--------------------------	------	--	----------------------	------	-----------------------------	----------------------	------	---	----------------------	------	---	----------------------	------	--	----------------------	------	-----------------------------------	----------------------	------	-----------------------------------	----------------------	------	-----------------------------------

The full list of tunable parameters is available via `tune_multiply.py --help`. Some parameters produce distinct code paths (e.g., SMM_BS=1 vs SMM_BS=2), so the parameter space is non-smooth.

Tuned Parameters

Pre-tuned parameter sets in `params/`:

File	Device		tune_multiply_PVC.csv	Intel Data Center GPU Max (PVC)	tune_multiply_BMG.csv	Intel Battlemage (BMG)	tune_multiply_A100.csv	NVIDIA A100	tune_multiply_H100.csv	NVIDIA H100	tune_multiply_GH200.csv	NVIDIA GH200	tune_multiply_V100.csv	NVIDIA V100	tune_multiply_P100.csv	NVIDIA P100	tune_multiply_Mi250.csv	AMD MI250
------	--------	--	-----------------------	---------------------------------	-----------------------	------------------------	------------------------	-------------	------------------------	-------------	-------------------------	--------------	------------------------	-------------	------------------------	-------------	-------------------------	-----------

Parameters are matched by device ID at runtime with best-match fallback. Single and double precision coexist in the same CSV.

```
## Use embedded parameters (default)
./acc_bench.x 5 30000 13 5 7
```

```
## Disable tuned parameters
OPENCL_LIBSMM_SMM_PARAMS=0 ./acc_bench.x 5 30000 13 5 7
```

```
## Load from a specific CSV file
OPENCL_LIBSMM_SMM_PARAMS=params/tune_multiply_PVC.csv ./acc_bench.x 5 30000 13 5 7
```

Rebuild with a specific device's parameters embedded:

```
make realclean
make WITH_GPU=PVC
```

Auto Tuning

Uses OpenTuner to explore the parameter space per (M, N, K) kernel. The tuner communicates with the benchmark driver through environment variables.

```
cd samples/smm
pip install -r requirements.txt
```

Build `acc_bench.x` before tuning. Keep GPU clocks and driver state as stable as practical during a run.

tune_multiply.py Use `tune_multiply.py` when you want direct control over one tuning run or over JSON maintenance.

Tune one kernel and write JSON results in the current directory:

```
./tune_multiply.py 13x5x7
```

Limit the search time, choose the JSON directory, and set the benchmark batch size:

```
mkdir -p params/local
./tune_multiply.py 13x5x7 --stop-after=300 -p params/local -s 30000
```

Tune several explicit kernels from a file, one $M \times N \times K$ per line:

```
printf '%s\n' 13x5x7 23x23x23 32x32x32 > kernels.txt
./tune_multiply.py kernels.txt --stop-after=180 -p params/local
```

Merge JSON files into a CSV file:

```
./tune_multiply.py -m -p params/local -o tune_multiply_local.csv
```

Update stored device names after rebuilding for a different target:

```
./tune_multiply.py -u -p params/local
```

Check existing JSONs without re-tuning:

```
./tune_multiply.py -c -p params/local
```

Useful options:

Option	Meaning		--stop-after N	Stop the search after N seconds		-p path	Directory for JSON input and output		-s size	Benchmark batch size, also called stack size		-a level	Tuning level: 0=all, 1=most, 2=some, 3=least		-m	Merge JSON files into a CSV file		-o file	CSV output file		-u [device]	Update JSON device names		-c [epsilon]	Validate JSON entries		-d	Delete outperformed duplicates during merge	
--------	---------	--	----------------	---------------------------------	--	---------	-------------------------------------	--	---------	--	--	----------	--	--	----	----------------------------------	--	---------	-----------------	--	-------------	--------------------------	--	--------------	-----------------------	--	----	---	--

The tuner can run under MPI. It detects local MPI rank variables and uses them to select `LIBXSTREAM_DEVICE`, so ranks on the same node can use different devices. This is most useful when each rank is tuning a different kernel.

```
mpirun \
./tune_multiply.py 13x5x7 --stop-after=300 -p params/local : \
./tune_multiply.py 23x23x23 --stop-after=300 -p params/local
```

Interrupted tuning writes the best result seen so far. If you tune a different kernel with `tune_multiply.py` directly, remove any old OpenTuner database first:

```
rm -rf opentuner.db
```

The wrapper below does this cleanup automatically.

tune_multiply.sh Use `tune_multiply.sh` when you want to expand triplet specifications, retune a directory, or split a larger tuning job into parts.

Tune a compact Cartesian-product specification:

```
./tune_multiply.sh -t 300 4 10 15, 6 7 8, 23
```

Triplets are comma-separated groups. The command above expands to all $M \times N \times K$ combinations with M in 4 10 15, N in 6 7 8, and $K=23$.

Tune an explicit list instead:

```
./tune_multiply.sh -t 300 1x1x1 2x2x2 13x5x7 23x23x23
```

Split the same work into four parts and run the second part:

```
./tune_multiply.sh -t 300 -j 4 -i 2 4 10 15, 6 7 8, 23
```

Retune every JSON file found in a directory:

```
./tune_multiply.sh -u -p params/local -t 300
```

Limit the generated work before splitting it:

```
./tune_multiply.sh -t 180 -r 4 32 -m 64 -n 100 \
4 8 16 32, 4 8 16 32, 4 8 16 32
```

Useful options:

| Option | Meaning | ||| | -t seconds | Time limit per kernel | | -p path | Directory for JSON files | | -s size | Benchmark batch size, also called stack size | | -a level | Tuning level: 0=all, 1=most, 2=some, 3=least | | -u | Retune JSON files found under -p | | -d | Ask the merge step to delete outperformed JSONs | | -c | Continue with the next kernel after an error | | -b | Tune triplets in reverse order | | -j parts | Total number of tuning parts | | -i index | Part to run, using 1-based numbering | | -r low high | Keep kernels with low $3 < MNK \leq \text{high}$ | | -m extent | Keep kernels with M , N , and K no larger than this | | -n count | Keep only the first count kernels | | -k id | Use a predefined triplet set |

Bulk Tuning with MPI Under MPI, the wrapper defaults `-j` to the MPI world size and `-i` to `rank + 1`. It also forwards a normalized local rank to `tune_multiply.py`, so the Python tuner can select a different device per local rank.

For most MPI launchers, this is enough:

```
mpirun -np 8 ./tune_multiply.sh -t 300 -p params/local \
4 10 15, 6 7 8, 23
```

You can still spell out parts explicitly when the launcher or scheduler needs it:

```
mpirun \
./tune_multiply.sh -t 300 -j 4 -i 1 -p params/local \
4 10 15, 6 7 8, 23 : \
./tune_multiply.sh -t 300 -j 4 -i 2 -p params/local \
4 10 15, 6 7 8, 23 : \
./tune_multiply.sh -t 300 -j 4 -i 3 -p params/local \
4 10 15, 6 7 8, 23 : \
./tune_multiply.sh -t 300 -j 4 -i 4 -p params/local \
4 10 15, 6 7 8, 23 \
>out.log 2>&1
```

To retune an existing JSON directory across eight MPI ranks:

```
mpirun -np 8 ./tune_multiply.sh -u -p params/local -t 300
```

Managing Tuned Parameters JSON files are the working format. CSV files are the deployable format. A typical retune flow is:

```
make realclean
make WITH_GPU=P100
mkdir -p params/p100
./tune_multiply.sh -u -p params/p100 -t 300
./tune_multiply.py -m -p params/p100 -o params/tune_multiply_P100.csv
```

Keep GPU driver state persistent during tuning (e.g., `nvidia-smi -pm ENABLED` on headless NVIDIA systems).